This is used to bind the socket descriptor *sockfd* with the address *addr*; and *addrlen* is its length.

This is called assigning a name to the socket. Next up is *listen()*:

```
int listen(int sockfd, int backlog);
```

The *listen()* call marks the socket referred to by *sockfd* as a passive socket—one that will be used to accept incoming connections. The socket type should be SOCK_STREAM or SOCK_SEQPACKET, i.e., a reliable connection. The *backlog* argument defines the maximum length of the queue of pending connections for *sockfd*. If the queue grows above this, connections will be refused by clients.

Next, let's enter an infinite loop to keep on serving the client requests. Here we have to create another socket descriptor for the client, by calling *accept*. Now, whatever is written to this descriptor is sent to the client, and whatever is read from this descriptor is the data the client sent to the server:

```
int accept(int sockfd, struct sockaddr *addr, socklen_t
*addrlen);
```

The *accept()* system call is used with socket types SOCK_STREAM and SOCK_SEQPACKET. It extracts the first connection request on the queue of pending connections for the listening socket *sockfd*, creates a new connected socket, and returns a new descriptor referring to that socket—in our program, *cfd*. The newly created socket is not in the listening state. The original socket *sockfd* is unaffected by this call. The *addr* argument contains the address of the remote machine to which we connect; since we don't know the client's address in advance, it's NULL here. The *addrlen* argument is the length of *addr*—so, again, this is NULL.

Then, let us read a character (sent by the client to the server) from the descriptor with *read()*, increment it, and write to the descriptor using *write()*, to send the character to the client. Then, let us close the descriptor by calling *close()*.

The error handling I chose is based on the knowledge that these functions return a negative number on failure; I use *perror()* to display an error number message.

The client

And now the client (*client.c*—again, this can be found on the *LFY* website at http://linuxforu.com/article_source_code/aug11/client.zip). This client sends a character to the server running on port 29008 (or any arbitrary port):

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main(int argc, char* argv[])
{
        int cfd;
        struct sockaddr_in addr;
        char ch='r';

        cfd=socket(AF_INET, SOCK_STREAM, 0);
        addr.sin_family=AF_INET;
        addr.sin_addr.s_addr=inet_addr("127.0.0.1"); /* Check for
server on loopback */
        addr.sin_port=htons(29008);
        if(connect(cfd, (struct sockaddr *)&addr,
        sizeof(addr))<0) {
                perror("connect error");
                return -1;
        }
        if(write(cfd, &ch, 1)<0) perror("write");
        if(read(cfd, &ch, 1)<0) perror("read");
        printf("\nReply from Server: %c\n\n",ch);
        close(cfd);
        return 0;
}
```

The flow of the client program is similar to the server. The first difference is that it specifies an Internet address for the server (the loop-back "*localhost*" address) in *sin_addr*. Next, instead of listening, it calls the *connect()* system call to connect the socket referred to by *sockfd* to the address specified by *addr*. The returned descriptor will be used to communicate to the specified address. Then, in the program, we have used *write()* and *read()* to send a character to the server, and receive a character, and then closed the descriptor

Running the programs

Compile the programs as follows:

```
$ cc server.c -o server
$ cc client.c -o client
```

Then, run them:

```
$ ./server &
$ ./client
```

Run each program in a different terminal for a better view. See Figure 1 for the server's output, and Figure 2 for the client's.

A good beginning, right? In the next article we'll rewrite both these programs for IPv6, and move further to UDP. And yes, FOSS rocks! END

By: Pankaj Tanwar a.k.a. Singularity

The author is a FOSS enthusiast, loves rock music and C, and can be reached at *pankaj.tux@gmail.com*.